

Curso de Go

Aula 20 - Arquivos, Logs e Tratamento de Erros

Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 **código_**^[3]
BRAZUCA





Nessa Aula nos Vamos Ver

- **Como trabalhar com arquivos em Go : criar, abrir, ler e escrever arquivos**
- **Como adicionar informações em arquivos existentes**
- **Como remover arquivos**
- **Como utilizar o pacote os**
- **Como utilizar defer para fechar arquivos corretamente**
- **O que são logs e para que servem**
- **Como utilizar o pacote log**
- **Como salvar logs em arquivos**
- **Como funciona o tipo error em Go**
- **Como identificar e tratar erros**
- **Como criar erros personalizados**
- **Como utilizar errors.New() e fmt.Errorf()**
- **As principais boas práticas para tratamento de erros em Go**
- **Parte Prática**



Arquivos, Logs e Erros

- **Aplicações reais precisam constantemente:**
- **Ler arquivos**
- **Criar arquivos**
- **Salvar informações**
- **Registrar eventos**
- **Detectar problemas**
- **Tratar erros**
- **Go possui bibliotecas nativas para essas operações.**
- **Principais pacotes:**
 - **os**
 - **log**
 - **errors**
 - **fmt**



O pacote os

- **O pacote os permite interagir com recursos do sistema operacional.**
- **Ele pode ser utilizado para:**
 - **Criar arquivos**
 - **Abrir arquivos**
 - **Ler arquivos**
 - **Escrever arquivos**
 - **Apagar arquivos**
 - **Criar diretórios**
 - **Trabalhar com variáveis de ambiente**
 - **Importação:**

```
import "os"
```



Criando um arquivo

- A função `os.Create()` cria um novo arquivo.

```
package main

import "os"

func main() {

    arquivo, err :=
os.Create("dados.txt")

    if err != nil {
        return
    }

    defer arquivo.Close()
}
```



O arquivo `dados.txt` será criado no diretório onde o programa estiver sendo executado.



Escrevendo em um arquivo

- Depois de criar o arquivo, podemos escrever dados nele.

```
package main

import "os"

func main() {

    arquivo, err :=
os.Create("dados.txt")

    if err != nil {
        return
    }

    defer arquivo.Close()

    arquivo.WriteString("Aprendendo
Go!\n")
}
```



O método `WriteString()` escreve uma string no arquivo.



O comando defer

- Ao trabalhar com arquivos, é importante fechá-los após sua utilização.

```
arquivo.Close()
```

- Uma prática comum em Go é utilizar:

```
defer arquivo.Close()
```

defer faz com que a função seja executada quando a função atual terminar. Isso ajuda a garantir que o arquivo seja fechado corretamente.



Lendo um arquivo

- Para arquivos pequenos podemos utilizar: `os.ReadFile()`

```
package main

import (
    "fmt"
    "os"
)

func main() {

    dados, err :=
os.ReadFile("dados.txt")

    if err != nil {
        fmt.Println("Erro:", err)
        return
    }

    fmt.Println(string(dados))
}
```



`ReadFile()` retorna os dados como um conjunto de bytes.



Adicionando informações ao arquivo

- Podemos abrir um arquivo sem apagar seu conteúdo anterior.

```
arquivo, err := os.OpenFile(  
    "dados.txt",  
  
    os.O_APPEND|os.O_WRONLY|os.O_CREATE,  
    0644,  
)
```

- Depois:

```
arquivo.WriteString("Nova informação\n")
```

O_APPEND adiciona dados ao final do arquivo.

O_WRONLY abre para escrita.

O_CREATE cria o arquivo caso ele não exista.



Excluindo arquivos

- Para remover um arquivo utilizamos: `os.Remove()`

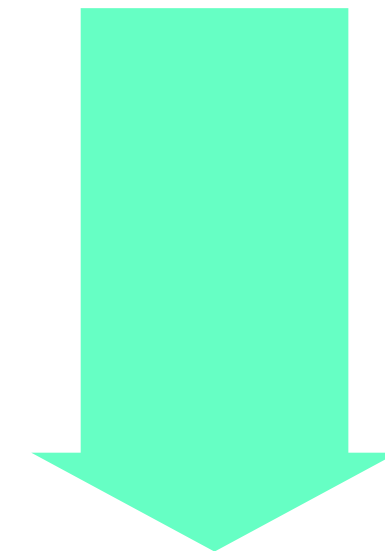
```
package main

import (
    "fmt"
    "os"
)

func main() {
    err := os.Remove("dados.txt")

    if err != nil {
        fmt.Println("Erro:", err)
        return
    }

    fmt.Println("Arquivo removido!")
}
```



Também existem funções para manipulação de diretórios, como:

- `os.Mkdir()`
- `os.MkdirAll()`
- `os.RemoveAll()`



O que são logs?

- **Logs são registros de eventos ocorridos durante a execução de uma aplicação.**
- **Exemplos**
 - **Servidor iniciado**
 - **Usuário autenticado**
 - **Arquivo criado**
 - **Conexão realizada**
 - **Erro ao acessar banco**
 - **Falha ao abrir arquivo**
- **Eles são importantes para:**
 - **Monitoramento**
 - **Auditoria**
 - **Debug**
 - **Segurança**
 - **Identificação de problemas**
- **Go possui o pacote padrão: log**



Utilizando log

- **Exemplo simples:**

```
package main

import "log"

func main() {

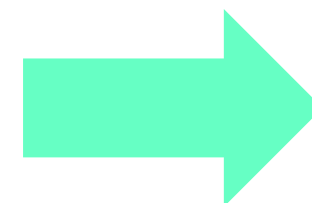
    log.Println("Aplicação iniciada")

    log.Println("Carregando
configurações")

    log.Println("Aplicação finalizada")
}
```

- **Saída aproximada:**

```
2026/08/09 10:30:15 Aplicação iniciada
2026/08/09 10:30:15 Carregando configurações
2026/08/09 10:30:15 Aplicação finalizada
```



O pacote log adiciona automaticamente data e horário.



Salvando logs em arquivo

- Podemos direcionar os logs para um arquivo.

```
package main
```

```
import (  
    "log"  
    "os"  
)
```

```
func main() {
```

```
    arquivo, err := os.OpenFile(  
        "app.log",  
        os.O_CREATE|os.O_APPEND|os.O_WRONLY,  
        0644,  
    )
```

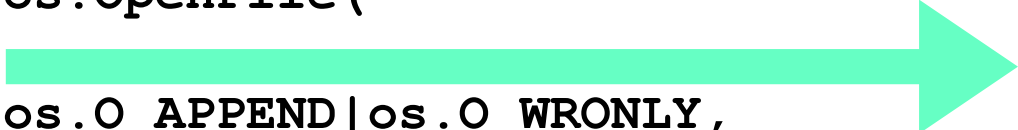
```
    if err != nil {  
        log.Fatal(err)  
    }
```

```
    defer arquivo.Close()
```

```
    log.SetOutput(arquivo)
```

```
    log.Println("Aplicação iniciada")
```

```
}
```



Agora os registros serão armazenados em:app.log



Tratando Erros em GO

- Em Go, os erros são tratados de forma explícita e simples. Em vez de utilizar exceções como try/catch, comuns em outras linguagens, Go utiliza valores do tipo error que são retornados pelas funções.
- O padrão mais comum é verificar se `err != nil`. Se isso acontecer, significa que ocorreu algum problema e o programa pode decidir o que fazer: exibir uma mensagem, registrar um log, retornar o erro ou encerrar a operação.
- Go utiliza valores do tipo: error
- É comum uma função retornar: `resultado, err := operacao()`
- Depois Verificamos :

```
if err != nil {  
    // ocorreu um erro  
}
```



- **Exemplo:**

```
arquivo, err :=  
os.Open("dados.txt")  
  
if err != nil {  
    fmt.Println("Erro:", err)  
    return  
}  
  
defer arquivo.Close()
```



Criando nossos próprios erros

- Podemos criar erros utilizando o pacote errors.

```
package main

import (
    "errors"
    "fmt"
)

func dividir(a, b float64) (float64, error) {

    if b == 0 {
        return 0, errors.New("divisão por zero")
    }

    return a / b, nil
}

func main() {

    resultado, err := dividir(10, 0)

    if err != nil {
        fmt.Println("Erro:", err)
        return
    }

    fmt.Println(resultado)
}
```



A função retorna dois valores:
resultado
erro

Quando não existe erro,
retornamos:
nil



Tratamento Idiomático de Erros

- Em Go, erros normalmente são tratados imediatamente.

```
resultado, err := operacao()
```

```
if err != nil {  
    return err  
}
```

- Também podemos adicionar contexto ao erro:

```
if err != nil {  
    return fmt.Errorf("erro ao abrir arquivo: %w", err)  
}
```




Adicionando contexto aos erros

- Em aplicações maiores, apenas retornar o erro original pode dificultar a identificação do problema. Podemos utilizar `fmt.Errorf()` para adicionar uma explicação antes de retornar o erro.

```
arquivo, err := os.Open("dados.txt")

if err != nil {
    return fmt.Errorf("erro ao abrir dados.txt: %w",
err)
}
```

- O `%w` mantém o erro original "dentro" do novo erro. Isso permite adicionar contexto sem perder a causa original do problema.
- Exemplo de mensagem:

```
erro ao abrir dados.txt: no such file or directory
```



errors.Is e errors.As

- Como um erro pode estar encapsulado dentro de outro, nem sempre devemos comparar erros diretamente. Para isso, Go oferece funções como `errors.Is()` e `errors.As()`.
- `errors.Is()` verifica se existe um determinado erro na cadeia de erros:

```
if errors.Is(err, os.ErrNotExist) {  
    fmt.Println("O arquivo não existe")  
}
```

- Já `errors.As()` é utilizado quando queremos verificar se existe um tipo específico de erro e acessar suas informações.

```
var pathErr *os.PathError  
  
if errors.As(err, &pathErr) {  
    fmt.Println("Arquivo:", pathErr.Path)  
}
```

• Resumo

`errors.Is()` → identifica um erro específico
`errors.As()` → identifica e acessa um tipo de erro
`fmt.Errorf() + %w` → adiciona contexto preservando o erro original



Boas práticas

- **Nunca ignorar erros importantes**
- **Tratar o erro próximo de onde ocorreu**
- **Retornar erros quando outra função deve tratá-los**
- **Adicionar contexto aos erros**
- **Usar %w ao encapsular erros**
- **Utilizar defer para fechar arquivos**
- **Utilizar logs para registrar eventos importantes**
- **Escreva mensagens que expliquem o que o programa estava tentando fazer quando o erro aconteceu.**

The screenshot shows a Go IDE with a file named 'package main Untitled-1'. The code defines a struct 'rect' with 'width' and 'height' of type 'int'. It includes two methods: 'area()' which returns the area, and 'perim()' which returns the perimeter. The 'main' function creates a 'rect' with width 10 and height 5, prints its area and perimeter, and then demonstrates calling these methods on a pointer to the 'rect'.

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de receptor
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

NÚCLEA
ACADEMY

Parte Prática

Arquivos, Logs e Tratamento de Erros